

Parallels-Utils

Parallels-Utils

Tagline: Compress, publish, and reuse your Parallels VM images across every machine.

Summary: Parallels-Utils is a Server Factory toolkit for managing Parallels (macOS) virtual-machine images: compressing and synchronizing image “matrices” used for development and testing, publishing them to a remote endpoint, and retrieving/running them on multiple workstations or servers. Usable standalone or as part of Server Factory.

Short description (~40 words): A Shell/Python toolkit for Parallels VM image lifecycle on macOS. It compresses and synchronizes Parallels images, publishes them to a remote endpoint, and retrieves and runs them across multiple computers — driven by simple settings files, usable independently or within Server Factory.

Long description (150-250 words): Parallels-Utils solves a practical DevOps problem for macOS-based development: teams build “matrices” of Parallels virtual machines (different OSes/configurations used for development and testing), and those images need to be compressed, published, retrieved, and run consistently across many machines. The toolkit provides exactly that lifecycle. A synchronization mechanism compresses Parallels images and keeps them in sync; a publishing mechanism uploads images to a remote endpoint; and a retrieval mechanism lets any workstation or server pull the published images and run them as VMs. Configuration is intentionally simple and file-driven: `image_location.settings` defines where images live on the filesystem, `image_provider.settings` defines the base URL for published images, and `image_sync.sh` defines the upload script — with examples shipped in an `Examples` directory. Operators use `publish_images.sh` to publish and `run.sh` to launch VMs. It requires Parallels for the appropriate macOS version and Python 3. The toolkit is designed to be dual-use: it can operate as part of a larger Server Factory project or completely independently, reflecting the org’s decoupling philosophy. It even ships a short video tutorial link. As part of the Server-Factory family, it

complements Qemu-Utils (the Linux/QEMU equivalent), giving the ecosystem VM-image management on both macOS/Parallels and cross-platform/QEMU backends.

Why we built it: Sharing consistent VM development/test environments across a team is tedious — images are large, and every machine needs the same matrix. Parallels-Utils automates compression, publishing, and retrieval so a canonical set of Parallels VMs is reproducible everywhere.

Why it's a game-changer: It turns heavyweight Parallels images into a publishable, syncable artifact set that any machine can pull and run — with trivial settings-file configuration and no dependency on the rest of Server Factory.

What's innovative: - Compression + synchronization of Parallels image “matrices” for dev/test. - Publish/retrieve workflow so images are reusable across many computers. - Settings-file-driven configuration (location/provider/sync) with shipped examples. - Dual-use: standalone or as a Server Factory component.

Biggest technical challenges + how solved: - *Large image distribution:* solved with compression plus a publish-to-remote-endpoint + retrieve workflow. - *Reproducibility across machines:* solved with provider/location settings so every host resolves the same image set. - *Ease of use:* solved with simple `publish_images.sh` / `run.sh` scripts and example settings files.

Tech stack (why + how): - **Shell** — the publish/run/sync scripts (primary language, ~5.3K bytes). - **Python 3** — supporting tooling (required dependency, ~3K bytes). - **Parallels (macOS)** — the virtualization backend being managed. - **Settings files (.settings)** — declarative configuration for location/provider/sync.

Public links: - GitHub: <https://github.com/Server-Factory/Parallels-Utils> (public; fork within the org). - Video tutorial: linked from the README (YouTube). - Part of the Server Factory ecosystem (references Core-Framework).

Suggested diagrams/illustrations (OpenDesign): 1. Image lifecycle: build → compress → publish → retrieve → run. 2. One provider endpoint → many workstations pulling the same image matrix. 3. Settings-file map (location/provider/sync) feeding the scripts.

Site relevance: vasic.digital (DevOps/virtualization tooling heritage). Niche, macOS-specific; present within the Server Factory toolchain group. Not AI-related.

Priority tier: serverfactory-tertiary

Source provenance: `gh repo view Server-Factory/Parallels-Utills`
README (goals, settings files, scripts, requirements, tutorial
link); `gh api repos/Server-Factory/Parallels-Utills/languages` (Shell/
Python); `_analysis/github-helix-others.md` (org listing).